

Taller — Complejidad algorítmica y Arreglos

Instrucciones generales

- Responde directamente sobre este mismo documento (puedes trabajarlo en Word, en Markdown o entregarlo como PDF).
- No es necesario programar en ningún lenguaje: basta con pseudocódigo, esquemas gráficos o explicaciones redactadas con tus propias palabras. Lo que se califica es el razonamiento, no la sintaxis de un lenguaje concreto.
- Cuando se solicite un esquema (arreglos, memoria), puedes elaborarlo a mano y anexar una foto, o apoyarte en una herramienta como draw.io.
- Sustenta siempre tus respuestas: una respuesta correcta sin argumentación no se considera completa.
- La fecha y el medio de entrega son los que indique el aula virtual o el repositorio del curso.

Parte 1 — Complejidad algorítmica (Big O)

1.1 Clasifica la complejidad

Para cada escenario, indica si corresponde a $O(1)$, $O(\log n)$, $O(n)$ u $O(n^2)$, justificando con tus propias palabras (sin necesidad de fórmulas matemáticas).

- *Buscar una palabra en un diccionario físico, abriéndolo siempre por la mitad de las páginas restantes.*

Respuesta: $O(\log n)$, ya que cada vez que abrimos el diccionario por la mitad, descartamos la otra mitad de las páginas que quedaban por revisar.

- *Revisar uno por uno todos los carnés de una caja hasta dar con el que se busca.*

Respuesta: $O(n)$, porque es necesario examinar, en el peor caso, cada uno de los elementos de la caja.

- *Tomar la primera carta de un mazo ya barajado.*

Respuesta: $O(1)$, pues tomar la carta de arriba demora lo mismo sin importar si el mazo tiene 52 cartas o muchísimas más.

- *Comparar a cada estudiante de un salón con todos los demás para ver si dos cumplen años el mismo día.*

Respuesta: $O(n^2)$, porque cada uno de los n estudiantes debe compararse nuevamente contra los otros n estudiantes; es una relación de parejas (n por n).

- *Consultar la hora en el reloj.*

Respuesta: $O(1)$, dado que es una lectura directa e inmediata que no depende de ninguna cantidad de datos.

1.2 De la vida real al análisis

Piensa en una actividad cotidiana (distinta a las vistas en clase) cuyo comportamiento sea $O(n)$: si la cantidad de elementos se duplica, el tiempo empleado también se duplica.

Respuesta: Tarea $O(n)$: lavar los platos a mano. Si normalmente lavo 5 platos y tardo 5 minutos, lavar 50 platos (10 veces más) me tomaría alrededor de 50 minutos (10 veces más tiempo): el esfuerzo crece de manera directamente proporcional y lineal a la cantidad de platos.

1.3 ¿Cuál escala mejor?

Existen dos maneras de resolver el mismo problema: la Forma A con $O(n^2)$ y la Forma B con $O(n \log n)$. Con pocos datos ($n = 5$) la Forma A resulta más veloz en la práctica. Aun así, ¿cuál convendría usar si se desconoce qué tan grande podrá llegar a ser n ?

Respuesta: Convendría usar la Forma B, $O(n \log n)$. Aunque con apenas 5 elementos la Forma A resulte más rápida (tal vez por tener menos operaciones internas), cuando n crece a cientos o miles, el tiempo de la Forma A se dispara rápidamente (por ejemplo, 1.000 elementos implicarían 1.000.000 de operaciones). La Forma B, en cambio, crece de forma mucho más moderada y controlada, garantizando que el programa siga respondiendo bien cuando el volumen de datos aumente.

1.4 Verdadero o falso (con justificación)

a) Un algoritmo $O(1)$ siempre es más veloz, en segundos reales, que uno $O(n)$.

Respuesta: Falso. La notación Big O no mide segundos, sino escalabilidad, es decir, cuántas operaciones adicionales se requieren a medida que n crece. Un algoritmo $O(1)$ puede tener un costo base muy alto (por ejemplo, tardar 5 minutos fijos en ejecutarse), mientras que un algoritmo $O(n)$ aplicado a un arreglo de solo 2 elementos puede resolverse en milisegundos.

b) La notación Big O describe cómo crece el tiempo de ejecución conforme aumentan los datos, no el tiempo exacto en segundos.

Respuesta: Verdadero. Big O describe la tendencia y la velocidad de crecimiento de las operaciones a medida que la entrada (n) tiende a infinito, dejando de lado el hardware, el lenguaje de programación o los segundos reales de ejecución.

c) Acceder por índice a un arreglo (`arreglo[5]`) es $O(n)$, porque hay que recorrer los primeros 5 elementos.

Respuesta: Falso. Acceder por índice a un arreglo es $O(1)$: el computador no recorre los elementos uno por uno, sino que calcula matemáticamente la dirección exacta en la memoria RAM y salta directamente a ella.

Parte 2 — Arreglos 1D

2.1 Diseña el arreglo

Se guardarán las calificaciones de 6 estudiantes de un curso en un arreglo llamado `notas`.

Esquema del arreglo con sus 6 casillas, mostrando el índice de cada una (el conteo inicia en 0):

VALOR	3,5	4,2	2,8	5,0	3,9	4,5
ÍNDICE	[0]	[1]	[2]	[3]	[4]	[5]

Si `notas = [3.5, 4.2, 2.8, 5.0, 3.9, 4.5]`, ¿qué valor y qué índice corresponden a la tercera nota ingresada?

Respuesta: Corresponde al valor 2.8, ubicado en el índice [2].

¿Cuál es la instrucción, en pseudocódigo, para acceder directamente a la nota del último estudiante sin recorrer el arreglo?

Respuesta: `Nota_Ultimo = notas[5]`

2.2 Direcciones de memoria

Supongamos que el arreglo notas del punto anterior se almacena en memoria RAM a partir de la dirección 0x2000, y que cada valor ocupa 4 bytes.

Cálculo de la dirección de memoria de notas[0], notas[3] y notas[5], aplicando: dirección = base + (índice × tamaño):

- notas[0]: $0x2000 + (0 \times 4) = 0x2000$
- notas[3]: $0x2000 + (3 \times 4) = 0x2000 + 12 \text{ bytes} = 0x200C$
- notas[5]: $0x2000 + (5 \times 4) = 0x2000 + 20 \text{ bytes} = 0x2014$

Explicación de por qué acceder a notas[3] es igual de rápido que acceder a notas[0], es decir, por qué en ambos casos es $O(1)$:

Respuesta: El computador aplica la fórmula de multiplicación y suma en un único ciclo para llegar directamente a esa dirección de memoria RAM; por eso da igual cuál posición se pida, siempre tardará lo mismo, ya que no recorre las posiciones una por una.

2.3 Búsqueda lineal vs. búsqueda binaria

Se tiene el arreglo ordenado edades = [15, 18, 20, 23, 27, 31, 35, 40].

Simulación paso a paso de una búsqueda lineal para hallar el valor 31. ¿Cuántas comparaciones se realizaron?

- Compara posición 0 (15) == 31? No.
- Compara posición 1 (18) == 31? No.
- Compara posición 2 (20) == 31? No.
- Compara posición 3 (23) == 31? No.
- Compara posición 4 (27) == 31? No.
- Compara posición 5 (31) == 31? Sí. En total, 6 comparaciones.

Simulación paso a paso de una búsqueda binaria para el mismo valor 31. ¿Cuántas comparaciones se realizaron?

- Centro entre los índices 0 y 7: $(0+7)/2 = 3.5 \rightarrow$ se compara con el índice 3 (23): $31 > 23$, así que se continúa por la mitad derecha (índices 4 a 7).
- Centro entre 4 y 7: $(4+7)/2 = 5.5 \rightarrow$ índice 5 (31): coincide, ¡encontrado! En total, 2 comparaciones.

¿Por qué la búsqueda binaria únicamente funciona si el arreglo ya está ordenado?

Respuesta: Porque su estrategia consiste en decidir hacia qué lado descartar datos con base en comparaciones. Si el arreglo estuviera desordenado, saber que 31 es mayor que 23 no garantizaría que 31 se encuentre a la derecha; podría estar en cualquier posición.

Si el arreglo tuviera 1 millón de elementos, ¿cuál de las dos búsquedas seguirías usando? Justifica comparando $O(n)$ frente a $O(\log n)$.

Respuesta: Seguiría usando la búsqueda binaria, $O(\log n)$. En el peor caso, la búsqueda lineal ($O(n)$) exigiría revisar hasta 1.000.000 de casillas, mientras que la binaria, al dividir el espacio de búsqueda entre 2 en cada paso, hallaría el dato en aproximadamente 20 comparaciones.

2.4 Insertar un elemento

Se tiene edades = [15, 18, 20, 23, 27] (5 casillas, sin espacio libre).

¿Qué tan costoso es insertar un nuevo valor al final si hay espacio disponible? ¿Y si no lo hay y debe crearse un arreglo más grande?

Respuesta: Si hay espacio disponible, la operación es muy económica, $O(1)$: basta con colocar el valor en la casilla libre. Si no hay espacio, resulta costosa, $O(n)$: hay que crear un arreglo nuevo de mayor tamaño y copiar allí, uno por uno, todos los elementos ya existentes.

¿Qué tan costoso es insertar el valor 21 en la mitad del arreglo, de modo que quede ordenado? ¿Qué debe hacerse con los demás elementos?

Respuesta: Es costoso, $O(n)$: es necesario desplazar hacia la derecha todos los elementos mayores que 21 para abrir el espacio donde se ubicará.

Comparación de ambos casos en notación Big O:

Respuesta: Insertar al final con espacio libre: $O(1)$. Insertar en la mitad, o tener que redimensionar el arreglo: $O(n)$.

Parte 3 — Arreglos 2D

3.1 Diseña la matriz

Se almacenará la disposición de un salón de clase de 3 filas por 4 columnas, donde cada casilla indica si el puesto está ocupado (1) o libre (0).

Esquema de la matriz salon con sus 3 filas y 4 columnas, indicando los índices [fila][columna] de cada casilla:

fila \ columna	columna 0	columna 1	columna 2	columna 3
fila 0	Salon[0][0]	Salon[0][1]	Salon[0][2]	Salon[0][3]
fila 1	Salon[1][0]	Salon[1][1]	Salon[1][2]	Salon[1][3]
fila 2	Salon[2][0]	Salon[2][1]	Salon[2][2]	Salon[2][3]

Instrucción en pseudocódigo para acceder al puesto de la fila 2, columna 3:

Respuesta: estado = salon[2][3]

¿Cuántas casillas tiene en total esta matriz, y cómo se calcula en general para filas $\times$ columnas?

Respuesta: Esta matriz tiene $3 \times 4 = 12$ casillas. Fórmula general: Total = filas $\times$ columnas.

3.2 De 2D a memoria (row-major)

La memoria RAM es en realidad una única fila continua de casillas: no existen físicamente 'filas y columnas' dentro de ella. Por esa razón, una matriz se almacena 'aplanada', recorriéndola fila por fila. Supongamos que el salón (3 filas $\times$ 4 columnas) se guarda a partir de la dirección 0x1000, con cada valor ocupando 4 bytes, en orden por filas (row-major).

1. Representación de las 12 casillas de salon aplanadas en una sola fila de memoria:

Respuesta: [0,0] [0,1] [0,2] [0,3] [1,0] [1,1] [1,2] [1,3] [2,0] [2,1] [2,2] [2,3]

2. Cálculo de la dirección de memoria de la casilla [1][2], usando: dirección = base + ((fila $\times$ número_de_columnas + columna) $\times$ tamaño):

Respuesta: Con base = 0x1000 (4096), filas = 3, columnas = 4 y tamaño = 4 bytes: dirección = $0x1000 + ((1 \times 4) + 2) \times 4 = 0x1000 + (4 + 2) \times 4 = 0x1000 + 24 \text{ bytes} = 0x1018$.

3. *Cálculo también de las direcciones de [0][0] y de [2][3] (la última casilla):*

Respuesta: [0][0]: $0x1000 + ((0 \times 4) + 0) \times 4 = 0x1000$. [2][3]: $0x1000 + ((2 \times 4) + 3) \times 4 = 0x1000 + (8 + 3) \times 4 = 0x1000 + 44 \text{ bytes} = 0x102C$.

3.3 Recorrido con ciclos anidados

Se busca contar cuántos puestos están ocupados (valor 1) en toda la matriz salon.

1. *Pseudocódigo del recorrido con dos ciclos anidados (uno para filas y otro para columnas):*

Respuesta: ocupados = 0 · Para f desde 0 hasta 2 (filas): Para c desde 0 hasta 3 (columnas): Si salon[f][c] == 1, entonces ocupados = ocupados + 1.

2. *Si la matriz tiene f filas y c columnas, ¿cuál es la complejidad de este recorrido en notación Big O? ¿Por qué?*

Respuesta: $O(n)$, donde n es el número total de casillas, ya que cada casilla debe revisarse individualmente, exactamente una vez.

3. *¿Qué ocurriría con el tiempo de ejecución si f y c se duplican simultáneamente?*

Respuesta: Si las filas se duplican (2f) y las columnas se duplican (2c), el total de casillas se multiplica por 4 ($2f \times 2c = 4fc$); por lo tanto, el tiempo de ejecución también se multiplicará aproximadamente por 4.

3.4 Conectando todo

Comprender la complejidad algorítmica (Big O) es lo que permite elegir adecuadamente entre estructuras y algoritmos para que un programa escale bien. Los arreglos, tanto 1D como 2D, resultan muy valiosos porque la memoria RAM los almacena de forma contigua, lo que hace que leer cualquier dato conociendo su índice sea inmediato, $O(1)$. Sin embargo, buscar información sin ningún criterio dentro de ellos es ineficiente, $O(n)$, cuando existen millones de datos. Ahí es donde se conecta con la lógica vista: si mantenemos los arreglos ordenados, podemos aplicar algoritmos tan eficientes como la búsqueda binaria, $O(\log n)$, lo que demuestra que combinar la estructura de datos correcta con el algoritmo óptimo es la clave real para procesar datos a gran escala.